

Concepts

DST

Deterministic simulation testing (DST) runs NautilusTrader under a seed-controlled runtime so that timing-sensitive execution behavior is bitwise reproducible from a single integer. This guide explains what DST is, how NautilusTrader supports it, what guarantees the support provides, and where those guarantees stop.

The goal is a published contract that external users and auditors can verify: the determinism NautilusTrader claims is backed by source-level evidence and enforced at commit time by a pre-commit hook that runs in continuous integration.

Introduction

What DST is

DST is a testing technique for concurrent systems. A single seed fully determines an execution, including task scheduling, timer firings, and random values. Two runs with the same seed, binary, and configuration produce identical observable behavior. When a property fails, the seed is the reproduction: the same seed replays the failure every time.

Scheduling decisions in an async runtime come from ambient process state: task wake order, timer resolution, thread scheduling, hash seeds. None of that is controlled by the test harness, which is why a race that surfaces once in CI is usually hard to reproduce on demand. DST replaces those ambient sources with a seeded pseudorandom sequence, so the interleaving is a function of the seed.

FoundationDB applied the pattern to a production distributed database starting around 2009; in the Rust ecosystem, [madsim](#) intercepts `tokio` primitives to provide a deterministic scheduler.

The bugs DST targets are the ones that escape unit, integration, property, and acceptance testing: channel wakeup ordering, drain races at shutdown, startup sequencing, reconciliation

ordering, recovery-path correctness. All involve interleavings that other test layers cannot exhaustively cover but a deterministic scheduler can explore systematically.

What this guide covers

NautilusTrader's DST support has two halves:

- **The contract:** what the runtime guarantees under seed-controlled execution, and under which conditions.
- **The enforcement:** the source-level seams that implement the contract and the pre-commit hook that keeps them in place.

Goals

- **Seed-reproducible execution** for the in-scope portion of the NautilusTrader runtime.
- **Honest scope.** The contract lists what is covered and what is not. No silent fallbacks to real wall-clock time or unseeded RNG; conditions that weaken the guarantee are enumerated.
- **Enforcement in source.** A pre-commit hook fails commits that add banned patterns to the DST path, so the contract stays true without relying on reviewer attention.
- **Minimum necessary instrumentation.** The seams route time, task scheduling, and randomness through a deterministic source only where the contract requires it; everything else runs unchanged.

Approach

`madsim` determinizes only the `tokio` primitives that route through its aliased submodules (`time`, `task`, `runtime`, `signal`). Wall-clock reads, monotonic reads, RNG draws, hash iteration, and `select!` polling bypass `tokio` entirely and need their own seams. Layer 1 swaps the aliased submodules for `madsim`; Layer 2 supplies the seams.

Layer 1: runtime swap

Under the `simulation` Cargo feature on `nautilus-common`, four `tokio` submodules are routed through `madsim` when `RUSTFLAGS="--cfg madsim"` is set:

- `time` (timers, intervals, monotonic `Instant`).
- `task` (spawning and joining async tasks).
- `runtime` (the runtime builder and handle).
- `signal` (process signals such as `ctrl_c`; re-export is available, call-site adoption is partial as noted under [scope boundaries](#)).

These re-exports live in `nautilus_common::live::dst`. DST-path call sites for `time`, `task`, and `runtime` import from this module rather than directly from `tokio`, so toggling the feature switches the async runtime in one place for the primitives that are fully routed. Under normal builds, the re-exports resolve to real `tokio`. Under `simulation` + `cfg(madsim)`, they resolve to `madsim`'s deterministic counterparts.

Everything else that `tokio` provides (`sync`, `io`, `select!` as a macro, `fs`, `net`) uses real `tokio` unconditionally. Transitive crates (`tokio-tungstenite`, `tokio-rustls`, `request`) are unaffected.

Layer 2: nondeterminism substitution

Nondeterminism outside the async runtime is redirected through explicit seams:

- **Wall-clock reads** go through `nautilus_core::time::duration_since_unix_epoch`. Under simulation this routes to `madsim::time::TimeHandle::try_current()`, preserving Unix-epoch semantics for order and fill timestamps. When called outside a madsim runtime (plain `#[rptest]` test bodies), it falls back to `SystemTime::now()`, which under `cfg(madsim)` is libc-intercepted to the same real syscall a normal build would use. Production paths under simulation always run inside a runtime, so they continue to receive virtual time.
- **Monotonic reads** go through `nautilus_common::live::dst::time::Instant`. The type resolves to `tokio::time::Instant` on normal builds (for compatibility with `tokio::test(start_paused)` test helpers) and `madsim::time::Instant` under simulation.
- **Network-local monotonic reads** go through `nautilus_network::dst::time`. The crate sits below `nautilus-common` in the dependency graph and exposes a local re-export module with the same semantics.
- **Hash iteration order** in the reconciliation manager and the order matching engine uses `IndexMap` and `IndexSet` rather than `AHashMap` and `AHashSet`. `AHash` randomizes its hasher per process; insertion-order iteration is needed where order drives downstream event publication or the sequence in which a seeded `FillModel` RNG is consumed.

- `tokio::select!` polling order uses the `biased;` modifier at every production site on the DST path. Unbiased `select!` polls branches in an order chosen by an unintercepted RNG.

Determinism contract

Under the conditions below, a run identified by `(seed, binary hash, configuration hash)` on the same platform produces bitwise-identical:

1. Scheduling order of async tasks.
2. Timer firings (virtual monotonic and virtual wall-clock).
3. RNG output from `madsim::rand`.
4. Channel delivery order on `tokio` primitives.

Required conditions

The contract holds only when all of the following are true:

1. The `simulation` Cargo feature is active and `RUSTFLAGS="--cfg madsim"` is set. Both are required. The feature activates the deterministic runtime; the cfg flag activates `madsim`'s libc-level `clock_gettime` and `getrandom` intercepts. One without the other silently falls back to real `tokio` and breaks determinism without an error.
2. Every `tokio::select!` call site on the DST path uses the `biased;` modifier.
3. Monotonic time reads route through the DST seam (either `nautilus_common::live::dst::time` or `nautilus_network::dst::time`), not `std::time::Instant::now` directly.
4. Wall-clock time reads route through `nautilus_core::time::duration_since_unix_epoch`.
5. Randomness routes through `madsim::rand`. `rand::thread_rng`, `rand::rng()`, `fastrand`, `getrandom`, and `OsRng` are not intercepted.
6. Iteration-order-sensitive collections use `IndexMap` or `IndexSet`, not `AHashMap` or `AHashSet`.
7. `tokio::task::LocalSet` construction is cfg-gated out under simulation. `madsim` does not provide `LocalSet`; `spawn_local` works without it.
8. `tokio::task::spawn_blocking` call sites are cfg-gated or removed. A blocking call escapes the deterministic scheduler.

Static enforcement

A pre-commit hook named `check-dst-conventions` enforces the structural conditions in source. The hook lives at `.pre-commit-hooks/check_dst_conventions.sh`, runs as part of the standard pre-commit suite, and runs in continuous integration. It covers the 16 in-scope workspace crates and fails the commit when it detects any of:

- Raw `std::time::Instant::now()` or `SystemTime::now()` reads, including bare forms when the enclosing file imports the type from `std::time`.
- Raw RNG usage (`rand::thread_rng`, `rand::rng()`, `fastrand::`, `getrandom::`, `OsRng`) without cfg gating.
- `tokio::select!` blocks missing `biased;` within the first three lines.
- `std::thread::spawn`, `std::thread::Builder::new`, or `tokio::task::spawn_blocking` calls that lack a preceding `#[cfg(test)]`, `#[cfg(not(madsim))]`, or `#[cfg(not(all(feature = "simulation", madsim)))]` attribute.
- `AHashMap` or `AHashSet` in iteration-order-sensitive files on the DST path. The full set of files is under audit; enforcement currently covers `crates/live/src/manager.rs` and `crates/execution/src/matching_engine/engine.rs`, and expands as further files are reviewed.
- Direct `tokio::net::TcpStream::connect` / `tokio::net::TcpListener::bind` reaches that bypass `nautilus_network::net`. The seam re-exports `tokio::net` types under normal builds and swaps to `turmoil::net` under the `turmoil` feature, so all TCP entry points share a single cfg-gated swap point.

The hook supports two exception forms:

- An inline `// dst-ok` marker on a specific line, typically accompanied by a short reason (for example, log-only wall-clock timing that does not affect state).
- A small file-level allowlist in the hook script itself for sites classified as leave-alone in the codebase audit (log timing in the cache module, progress reporting in the DeFi module).

Test files, files under `tests/`, `python/`, and `ffi/` directories, and lines inside an inline `#[cfg(test)]` module are excluded because they are not part of the DST path.

In-scope crates

The hook applies to the 16 workspace crates in the transitive closure of `nautilus-live`:

- `analysis`, `common`, `core`, `cryptography`, `data`, `execution`, `indicators`, `live`, `model`, `network`, `persistence`, `portfolio`, `risk`, `serialization`, `system`, `trading`.

Adapter crates and infrastructure crates (Redis, Postgres) are out of scope. Their DST suitability requires a separate audit before they enter the DST path.

Implementation notes

Concrete changes the DST audit produced in this repository. Use this as the starting point when investigating whether a code path is on the DST path and how it routes today.

Iteration-order seams

Production sites where `AHashMap` / `AHashSet` flipped to `IndexMap` / `IndexSet` because the iteration order is observable on the DST path:

- **Matching engine** (`crates/execution/src/matching_engine/engine.rs`): nine fields (`execution_bar_types`, `execution_bar_deltas`, `account_ids`, `cached_filled_qty`, `bid_consumption`, `ask_consumption`, `queue_ahead`, `queue_excess`, `queue_pending`). Iterated removes use `.shift_remove()`. Closes [#3914](#).
- **Reconciliation manager** (`crates/live/src/manager.rs`): hook-enforced, plus `ReconciliationResult.orders` and `ReconciliationResult.fills`.
- **Account trait** (`crates/model/src/accounts/`): `balances`, `balances_total`, `balances_free`, `balances_locked`, `starting_balances` returns. Storage fields on `BaseAccount` and `MarginAccount` are `IndexMap`.
- **Position events** (`crates/model/src/position.rs`): `Position::commissions` flipped to `IndexMap` (consumed via `.values()` in `events/position/snapshot.rs`).
- **Portfolio aggregation** (`crates/portfolio/src/portfolio.rs`): `unrealized_pnls`, `realized_pnls`, `net_positions` storage; `accumulate_mark_values` builds `IndexMap<Currency, f64>`.
- **Data engine** (`crates/data/src/engine/`): `book_snapshot_counts`, `bar_aggregators`, `BookSnapshotInfos`. Iterated removes use `.shift_remove()`.

- **Execution engine** (`crates/execution/src/engine/`): `ExecutionEngine.clients`, plus the `client_ids` / `venues` accumulators in `get_clients_for_orders()`.
- **Trading algorithm** (`crates/trading/src/algorithm/core.rs`): `strategy_event_handlers` (drives ordered `msgbus::unsubscribe_*` fan-out).
- **Analyzer** (`crates/analysis/src/analyzer.rs`): `account_balances`, `account_balances_starting`.
- **Cache API** (`crates/common/src/cache/mod.rs`): `get_orders_for_ids` and `get_positions_for_ids` sort their `Vec` returns by `client_order_id` / `position_id` before returning. Storage stays on `AHashSet` (set semantics).

Remaining `AHashMap` / `AHashSet` sites in the in-scope crates are lookup-only, behind concurrent shared-ownership wrappers (`Arc<DashMap>`, `AtomicMap`), or feed into commutative aggregation. Any new in-scope site that drives observable iteration order is a regression that the per-area audit guards against.

Time seams

`Instant::now` / `SystemTime::now` call sites that remain on the DST path are either inside `# [cfg(test)]`, file-allowlisted in the hook, or carry an inline `// dst-ok` marker with a reason:

- `crates/common/src/testing.rs:81,108` `wait_until` / `wait_until_async`
- `crates/execution/src/engine/mod.rs:822,847` init log timing
- `crates/common/src/cache/mod.rs:569,904,3895` log and audit timing (file-allowlisted)
- `crates/model/src/defi/reporting.rs:59,123` progress logging (file-allowlisted)
- `crates/core/src/time.rs` seam definition site (file-allowlisted)

`chrono::Utc::now` is hook-banned in the in-scope crates. The remaining call sites are the logging bridge and writer (scoped out under "Logging runs on real OS threads"). The `crates/core/src/datetime.rs::is_within_last_24_hours` helper used to reach `chrono::Utc::now` from non-logging paths; it now routes through `nautilus_core::time::nanos_since_unix_epoch()` and compares in `u64` nanos directly.

Randomness seams

Production RNG sites on the DST path:

- `crates/core/src/uuid.rs::UUID4::new()` routes through `madsim::rand::thread_rng()` when called inside a madsim runtime under simulation, falling back to `rand::rng()` outside one (and on normal builds). Production paths under simulation always run inside a runtime, so they consume seeded bytes; plain `#[rtest]` tests under `cfg(madsim)` use the host RNG. Reachable from order and event factories in `nautilus-common` and `nautilus-risk`.
- `crates/execution/src/models/fill.rs::default_std_rng()` routes the same way. Called from `ProbabilisticFillState::new()` when no seed is provided. With a seed, `StdRng::seed_from_u64` is deterministic by construction.
- `crates/execution/src/matching_engine/ids_generator.rs:167,179` USES `nautilus_core::UUID4::new()` for the `use_random_ids` path. The default ID scheme (`{venue}-{raw_id}-{count}`) is deterministic without it.

Allowed-with-marker: `crates/network/src/backoff.rs:105` for reconnect jitter, `// dst-ok` (transport layer).

Tokio submodule split

`madsim` aliases `time`, `task`, `runtime`, and `signal`. Other tokio submodules (`sync`, `io`, `select!`, `fs`, `net`) stay on real tokio under simulation. Extending the swap further would require rebuilding `tokio-tungstenite`, `tokio-rustls`, and `request` against shimmed `tokio::net::TcpStream`, which the audit ruled out as too invasive.

In-scope sites that touch real `tokio::net` / `tokio::io` directly:

- `crates/network/src/net.rs:37` re-exports `tokio::net::{TcpListener, TcpStream}`
- `crates/network/src/socket/client.rs:46,356` `tokio::io::{AsyncReadExt, AsyncWriteExt}`
- `crates/network/src/tls.rs:22` `tokio::io::{AsyncRead, AsyncWrite}`
- `crates/network/src/websocket/types.rs:26,29` aliases `MaybeTlsStream<tokio::net::TcpStream>`

These run on real sockets even under simulation. Channel delivery order on `tokio::sync` stays deterministic because the sender and receiver tasks are scheduled by the madsim executor even though the channel implementation is real.

Raw thread escape rules

Rule 4 of the hook bans raw thread spawning outside three escape cases:

- `#[cfg(test)]` test modules.
- `#[cfg(not(madsim))]` OR `#[cfg(not(all(feature = "simulation", madsim)))]` production sites (e.g. the logging writer thread).
- An inline `// dst-ok` marker.

`tokio::task::LocalSet` and `tokio::task::spawn_blocking` are not supported under `madsim`. The codebase audit found no production sites for either inside the in-scope crates; new sites must carry a cfg gate or `// dst-ok` marker.

Logging tests under simulation

The logging writer thread is cfg-gated out under simulation; under `cfg(madsim)` log events are dropped. Tests that init the file-logging writer would either hang or assert against an empty log file, so the affected submodules are gated out at the module boundary:

- `crates/common/src/logging/logger.rs::tests::serial_tests` (eight tests).
- `crates/common/src/logging/macros.rs::tests` (two tests).

`logger.rs::tests::sim_tests::test_init_under_madsim_skips_writer_thread_and_forces_bypass` runs under simulation and pins the gated behaviour.

Scope boundaries

The contract is deliberately narrow. The following weakenings are explicit, not oversights.

Python is not in DST scope

DST runs under a native Rust test harness. No Python interpreter starts during a DST run. The PyO3 bindings under `crates/*/src/python/`, the `ffi/` directories, and the Python packages under `nautilus_trader/` are excluded from the contract as a policy, not as a weakness. Any code reachable only from Python call paths is out of scope; any Rust path reachable from the native DST harness must satisfy the contract even if the same type is also exported to Python.

The `check-dst-conventions` hook encodes this policy by skipping `/python/` and `/ffi/` paths in the in-scope crates. Clock, RNG, and threading call sites behind those paths do not apply to

the contract.

The primary objective of DST is reliability of the Rust engine itself: the order lifecycle, reconciliation, matching, risk, and execution state machines. Deterministic replay of user strategies is a later, secondary goal that becomes available once strategies are authored in Rust or run through a Rust-native test harness. In the meantime, a Python strategy that calls `time.time()`, issues arbitrary network requests, or relies on thread scheduling can vary its command stream between runs; the Rust core will process the varying stream deterministically, but end-to-end replay from a Python entry point is not guaranteed.

Platform-scoped

`madsim`'s libc overrides for `clock_gettime` and `getrandom` are platform-specific. Cross-platform bitwise reproducibility is not claimed. A seed that reproduces a failure on Linux x86_64 may not reproduce on macOS aarch64.

Non-aliased dependencies escape silently

Any dependency that reaches the OS through a non-aliased path (direct `libc` calls, `std::net` bypass, crates using `fastrand` or `OsRng`) escapes the simulator without raising an error. The in-scope crates have been audited; adapter crates and infrastructure crates require their own audits before entering the DST path.

Transport-layer I/O is not simulated

`tokio-tungstenite`, `tokio-rustls`, `request`, `redis`, and `sqlx` use real `tokio` internally. Under simulation, WebSocket and HTTP I/O run on real networking. This is intentional: the initial target is order lifecycle determinism, not transport fault injection. Transport-layer determinism would require per-crate `madsim` shims that do not exist at this time.

Test modules that drive real localhost sockets (`crates/network/src/socket/client.rs::tests`, `::rust_tests`; `crates/network/src/websocket/client.rs::tests`, `::rust_tests`; `crates/network/tests/websocket_proxy.rs`) are cfg-gated out under `all(feature = "simulation", madsim)` because their production code paths reach `dst::time::*` (`madsim` time primitives), which panic when called from a `#[tokio::test]` runtime. The retry test modules (`crates/network/src/retry.rs::tests`, `::proptest_tests`) run under simulation: each test attribute is `cfg_attr`-swapped between `#[tokio::test(start_paused = true)]` and `#[madsim::test]`, time

reads and sleeps route through `crate::dst::time`, and explicit virtual-time advances go through a `cfg`-gated `advance_clock` helper so the same body covers both runtimes.

Signal handling

`nautilus_common::live::dst::signal` exposes a routed `ctrl_c` re-export. The `crates/live/src/node.rs` run loop routes through it, so node shutdown driven by `ctrl_c` is injectable from test code under `cfg(madsim)` via `madsim::runtime::Handle::send_ctrl_c`. Adapter-bin entry points still call `tokio::signal::ctrl_c` directly and remain scoped out.

Logging runs on real OS threads

The logging subsystem spawns a writer thread via `std::thread::Builder` and uses `std::sync::mpsc`. Under simulation, the thread is not spawned and log events are dropped. Log output is outside the determinism contract: the writer only writes, never reads or mutates simulation state.

Adapters

Adapter crates are out of scope for the initial DST contract. Each adapter has its own set of `chrono::Utc::now`, `SystemTime::now`, `Uuid::new_v4`, and transport-layer call sites. An adapter that enters the DST path must be audited for direct clock, RNG, and transport usage before its behavior can be covered by the contract.

Process-global lazy state consumes RNG bytes on first call

The contract holds within a single run-of-the-runtime. A few process-global lazy initialisations consume RNG bytes on first call, which matters for harnesses that run two seeded executions inside one process and compare traces.

- The `Ustr::from()` interner allocates and seeds its internal map on first use.
- `ahash::RandomState` seeds itself via `getrandom` (which `madsim` hooks under `cfg(madsim)`) on first instance creation.

Single-runtime tests (one body invocation per seed, fresh process) are unaffected: the consumption is part of the seeded execution and reproduces deterministically.

Same-seed equality frameworks that invoke the body twice in one process to diff the traces will see drift between runs. Run 1 pays the lazy-init cost and consumes RNG bytes; run 2 inherits the warm state and starts at a different offset in the RNG sequence.

Workaround: pre-warm process-global state outside the runtime before the comparison, for example by calling `Ustr::from("")` and constructing one `ahash::RandomState` once at process start. Both runs then start with the warmed state and consume the RNG sequence identically.

Adapter factories no longer expose `Rc<RefCell<Cache>>`

Commit `f0ea66da15` ("Standardize adapter cache access via `CacheView`") replaced the mutable `Rc<RefCell<Cache>>` parameter on `DataClientFactory::create` and `ExecutionClientFactory::create` with a `CacheView`. `CacheView` exposes `borrow()` for read access but does not expose the inner `Rc` handle.

This blocks DST-style harness factories that need to construct an `OrderMatchingEngine` inline inside the factory, because `OrderMatchingEngine::new` still takes `Rc<RefCell<Cache>>` and there is no public accessor to recover that handle from a `CacheView`.

Workarounds:

- Pin the harness consumer to a commit before `f0ea66da15`.
- Refactor the harness so it builds the `OrderMatchingEngine` outside the factory (where the kernel `cache` handle is still reachable) and passes the constructed engine into the client.

A longer-term escape hatch would be either an accessor on `CacheView` for the inner handle or an alternative `OrderMatchingEngine` constructor that accepts a `CacheView`. Neither is in the tree today.

Relationship to other testing layers

DST complements existing testing; it does not replace any of it.

Layer	Covers	DST relationship
Unit tests	Pure logic, calculations, parsers, transformers.	Unchanged.

Layer	Covers	DST relationship
Integration tests	Component interaction, I/O boundaries.	Unchanged. DST runs alongside, not in place of.
Property-based tests	Invariants over input domains (parsers, roundtrips).	Unchanged.
Acceptance tests	End-to-end backtest and live scenarios.	Unchanged.
Deterministic sim (DST)	Async timing, scheduling, recovery correctness.	Adds seed-replayable exploration.

DST's unique value is in the intersection of async concurrency and state-machine correctness. Bugs such as "a message at shutdown is dropped under a specific wakeup ordering" or "a reconciliation event is lost when iteration order reverses" are the target class. For anything else, the pre-existing test layers are the right tool.

Status

As of the current state of this repository:

- Layer 1 (runtime swap) is implemented. `nautilus_common::live::dst` exposes routed re-exports for `time`, `task`, `runtime`, and `signal`. Production call sites for `time`, `task`, and `runtime` route through the seam; signal call-site adoption is partial (see "Signal handling" under "Scope boundaries").
- Layer 2 (nondeterminism substitution) is implemented across the 16 in-scope crates. Seams exist for wall-clock time, monotonic time, randomness, and iteration order. The audit closures and remaining allowed call sites are enumerated under "Implementation notes".
- Static enforcement via `check-dst-conventions` is active in pre-commit and CI. The hook covers the load-bearing conditions; the `// dst-ok` marker convention permits per-line exceptions when justified.
- Build-and-test smoke gate under `cfg(madsim)` runs via the `dst` workflow (`.github/workflows/dst.yml`, invokes `make cargo-test-sim`). It compiles the in-scope crates with

`--features simulation` and runs every test that is sim-compatible today. Crates that consume `nautilus-model` types (`nautilus-common`, `nautilus-execution`) also run a second leg with `--features "simulation,high-precision"` so the seam-routed code paths are exercised under both fixed-point widths (`QuantityRaw` / `PriceRaw` as `u64` vs `u128`).

- All of `nautilus-common`. This leg compiles with `nautilus-core/simulation` propagated, so the explicit `wall_clock_now` cfg branch is selected for every test in the suite. Plain `#[rptest]` tests run outside a madsim runtime and route through the seam's `SystemTime::now()` fallback (the same path madsim's libc shim takes outside a runtime). The `live::dst::tests::test_dst_wall_clock_advances_with_virtual_time` test in this leg uses `#[madsim::test]` and asserts that `nanos_since_unix_epoch` advances with `madsim::time::sleep`, so virtual wall-clock behavior is validated end-to-end on the common leg.
- All of `nautilus-network` (transport-bound test modules are gated out at the source). Includes the seam pinning tests for sleep / timeout virtual time and the rate-limiter, plus the retry suites that exercise backoff timing under virtual time.
- All of `nautilus-execution`. The matching engine, fill model, and execution-engine state machines run under the deterministic scheduler with the seeded RNG.
- The cross-crate seam pinning tests in `nautilus-core` (`wall_clock_now` virtual time). Each leg runs with its own crate's `--features simulation` and uses `#[madsim::test]` where applicable, so the explicit cfg branches and virtual time are both validated.

Together this catches drift in the cfg-gated DST seams and exercises the in-scope state machines under the deterministic scheduler; it does not yet exercise determinism end-to-end.

- End-to-end runtime verification (same-seed diff over an in-scope code path) is out of scope for this repository. The structural conditions (Rule 1 to Rule 6) are enforced; the claim that a seed reproduces identical observable behavior across runs is plausible from the seam design but is not yet verified by a regression gate.

Further reading

- `.pre-commit-hooks/check_dst_conventions.sh` defines the five enforcement rules in full and documents the `// dst-ok` marker convention.

- External references: the [FoundationDB testing philosophy](#), the [TigerBeetle simulation testing blog posts](#), and the [madsim repository](#) for the deterministic runtime.

[< Rust](#)[Previous Page](#)[How-To >](#)[Next Page](#)